

06-01-2026

Lab - 7

Do all pre processing on diabetes csv

```
In [ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Import diabetes.csv data ser using sklearn

```
In [ ]: df = pd.read_csv('diabetes.csv')
df
```

Check the distribution of the target

```
In [ ]: df['Outcome'].value_counts()
```

Replace zeros with NaN for specific columns

```
In [ ]: column_to_replace = ['Glucose', 'BloodPressure', 'SkinThickness', 'BMI', 'Insulin']
df[column_to_replace] = df[column_to_replace].replace(0, np.nan)
```

Check for missing values

```
In [ ]: df.isnull().sum()
```

Fill missing values with median

```
In [ ]: cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
```

```
In [ ]: for col in cols:  
        df[col] = df[col].fillna(df[col].median())
```

```
In [ ]: df.isnull().sum()
```

Convert Targer data into interger code

```
In [ ]: df['Outcome'] = df['Outcome'].map({'Non Diabetic' : 0, 'Diabetic' : 1})  
df['Outcome']
```

Visualize Distributions

```
In [ ]: sns.pairplot(df, hue='Outcome')
```

```
In [ ]: sns.heatmap(df.corr(), annot=True, cmap='coolwarm')  
plt.show()
```

```
In [ ]: df
```

Divide the data into input and output

```
In [ ]: x = df.drop('Outcome', axis=1)
```

```
In [ ]: x.head()
```

```
In [ ]: y = df['Outcome']
```

```
In [ ]: y.head()
```

import KNN Model

```
In [ ]: from sklearn.neighbors import KNeighborsClassifier  
from sklearn.model_selection import train_test_split
```

Convert all features into Z score

```
In [ ]: from sklearn.preprocessing import StandardScaler
```

```
In [ ]: scaler = StandardScaler()
```

```
In [ ]: x_scaled = scaler.fit_transform(x)
```

```
In [ ]: x_scaled
```

Splitting the dataset into the Training set and Test set

```
In [ ]: x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.3, random_s
```

Create KNN Model

```
In [ ]: model = KNeighborsClassifier(n_neighbors=100)
```

Fitting KNN on dataset

```
In [ ]: model.fit(x_train, y_train)
```

```
In [ ]: model.score(x_test, y_test)
```

Display classification_report

```
In [ ]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_
```

```
In [ ]: y_pred = model.predict(x_test)
```

```
In [ ]: cm = confusion_matrix(y_test, y_pred)
```

```
In [ ]: display = ConfusionMatrixDisplay(cm, display_labels=['Diabetic', 'Non Diabetic'])
```

```
In [ ]: display.plot()
```

```
In [ ]: print(classification_report(y_test, y_pred))
```

Use Cross Validation to find best hyperparameter for KNN

```
In [ ]: from sklearn.model_selection import cross_val_score
```

```
In [ ]: from sklearn.model_selection import KFold
```

```
In [ ]: k = KFold(n_splits=10)
```

```
In [ ]: for train_index, test_index in k.split(x_scaled):  
        x_train, x_test = x_scaled[train_index], x_scaled[test_index]  
        y_train, y_test = y[train_index], y[test_index]  
        model1 = KNeighborsClassifier()  
        model1.fit(x_train, y_train)  
        print(model1.score(x_test, y_test))
```

```
In [ ]: cross_val_score(model, x_scaled, y, cv=k)
```

```
In [ ]: import numpy as np
```

```
In [ ]: x1 = np.arange(1,11).reshape(-1,1)
```

```
In [ ]: y1 = np.array([0,1,0,1,0,1,0,1,0,1])
```

```
In [ ]: x1
```

```
In [ ]: y1
```

```
In [ ]: for i,j in k.split(x1):  
        print("<----->")  
        print(x1[i])  
        print(x1[j])  
        x_train, x_test = x1[i], x1[j]  
        y_train, y_test = y1[i], y1[j]
```

```
In [ ]: parameters = {'n_neighbors':(1,2,3,4,5,6,7,8,9,10,11), 'weights':('uniform', 'distan
```

```
In [ ]: from sklearn.model_selection import GridSearchCV
```

```
In [ ]: m1 = KNeighborsClassifier()
```

```
In [ ]: gs = GridSearchCV(m1, parameters)
```

```
In [ ]: gs.fit(x_scaled, y)
```

```
In [ ]: gs.best_params_
```

```
In [ ]: gs.best_score_
```

```
In [ ]: gs.best_estimator_
```

```
In [ ]: gs.cv_results_
```